Task Manager: React frontend

The React app for the Task Manager project. It talks to the Express API in `../server`.

Built with React 18, React Router 6 and Vite. Plain CSS, no UI library.

Run it locally

From the repo root, `npm install` once, then `npm run dev` starts the API (port 3001) and this app together. To run only the frontend: `npm run dev:client` from the root, or `npm run dev` in this folder.

Open http://localhost:5173 and log in with a seeded account (`alice` / `alice123`, or `admin` / `admin123`).

In development, Vite forwards every `/api` request to `http://localhost:3001` (see `vite.config.js`), so you don't need CORS set up to work locally.

Environment variables

Copy `.env.example` to `.env`.

Variable	When	Value
`VITE_API_URL`	Production only	The deployed API, for example `https://your-api.onrender.com`. Leave empty locally.

Pages

Route	Page	Access
`/`	Landing	Everyone
`/login`	Log in (username or email, "Keep me logged in")	Logged out
`/register`	Sign up with live password rules	Logged out
`/dashboard`	Task list: search, status chips, priority filter, sort, mark done, delete	Logged in
`/board`	Board view: To do, In progress, Done columns	Logged in
`/tasks/new`	Create task with a live preview	Logged in
`/tasks/:taskId`	Edit or delete a task	Logged in
`/profile`	Account details and change password	Logged in
`/admin`	Manage all users and tasks	Admins only

Folder structure

```
src/
  api/          client.js (fetch wrapper, token storage, errors) and index.js (one
```

```
function per endpoint)
  components/   layouts, route guards, task form, badges, icons
  context/      AuthContext (who is logged in) and ToastContext (success and error
messages)
  pages/        one file per route
  utils/        dates, validation, hooks
  config.js     status and priority values
  styles.css    all styles, with color tokens at the top
```

What the frontend expects from the API

Everything API-related lives in `src/api/`, so if the backend differs, that's the only place to change.

Status and priority values are set in `src/config.js`: `todo`, `in-progress`, `done` and `low`, `medium`, `high`. If the Task model uses other strings, change them there.

Endpoints used

Method	Path	Used by
POST	`/api/auth/login`	Log in. Body: `{ username, password, rememberMe }`. `username` may hold an email.
POST	`/api/auth/register`	Sign up. Body: `{ username, email, password }`
GET	`/api/auth/me`	Restores the session on page load
GET	`/api/auth/profile`	Profile page
PUT	`/api/auth/profile`	Save username and email (team card B-7)
PUT	`/api/auth/password`	Body: `{ currentPassword, newPassword }` (team card B-7)
GET	`/api/tasks`	Query: `status`, `priority`, `search`, `sortBy` (`dueDate` or `createdAt`), `order` (`asc` or `desc`)
POST	`/api/tasks`	Body: `{ title, description, status, priority, dueDate }`
GET, PUT, DELETE	`/api/tasks/:taskId`	Edit page, mark done, board moves, delete
GET	`/api/admin/users`, `/api/admin/tasks`	Admin page
DELETE	`/api/admin/users/:userId`, `/api/admin/tasks/:taskId`	Admin page

Response shapes it accepts

- Login and register: `{ token, user }`. If register returns no token, the app logs in right after.
- Lists: a bare array, or `{ tasks: [...] }` / `{ users: [...] }` / `{ data: [...] }`.
- Single items: the object itself, or `{ task }` / `{ user }`.

- Errors: `{ message }` plus optional field errors as `{ errors: { field: "message" } }` or express-validator's `{ errors: [{ path, msg }] }`. Field errors show under the matching input.
- A 401 on any request logs the user out. A 429 shows the rate-limit message.

Security notes

- The JWT is sent as `Authorization: Bearer <token>`. It's kept in `localStorage` when "Keep me logged in" is checked, otherwise in `sessionStorage` (cleared when the browser closes).
- React escapes all text it renders, and the app never uses `dangerouslySetInnerHTML`, which guards against XSS.
- Every form validates on the client too, but the server's validation is the one that counts.

Deploy to Render (static site)

1. New, then Static Site, then connect the repo.
2. Root directory: `client`
3. Build command: `npm install && npm run build`
4. Publish directory: `dist`
5. Environment: `VITE_API_URL` = your deployed API URL
6. Redirects/Rewrites: add a rewrite from `/*` to `/index.html`, so refreshing on a page like `/dashboard` works.
7. On the API, set `CLIENT_URL` to the static site's URL so CORS allows it.

On Netlify, `public/_redirects` already handles step 6.